

**OBJECT-ORIENTED
PROGRAMMING
CL - 1004**

LAB 03

Fundamentals of Object-Oriented
Programming, Classes, Objects,
Access Modifiers, Setter and Getters
and Header Files

National University of Computer and Emerging Sciences–NUCES – Karachi

Instructor: **Talha Shahid**

Contents

1. Classes & Objects.....	3
1.1. Class	3
1.2. Class Name	3
1.3. Data Members	3
1.4. Member Functions.....	4
1.5. Objects	4
1.6. Accessing Public Data Members	4
1.8. Member Functions in Classes.....	6
1.8.1. Inside class definition	6
1.8.2. Outside Class Definition.....	6
2. Structure vs Classes	7
3. Transformation from Procedural to Object-Oriented Programming.....	7
3.1. Procedural Approach	7
3.2. Object-Oriented Approach.....	8
3.3. Object-Oriented Approach (another example with UML Diagram).....	10
4. Header Files.....	11
4.1. Essentials of Header Files	11
4.1.1. Purpose of Header Files.....	11
4.1.2. Header Guards.....	11
4.1.3. Example of a Header Guard.....	12
4.2. Creating and Using a Header File	12
4.2.1. Creating a Header File	12
4.2.2. Implementing the Header File	13
4.2.3. Using the Header File in Another Program	13

1. Classes & Objects

1.1. Class

A class is a programmer-defined data type that describes what an object of the class will look like when it is created. It consists of a set of variables and a set of functions.

We can think of class as a sketch (prototype) of a house. It contains all the details about the floors, doors, windows etc. Based on these descriptions we build the house. House is the object.

As, many houses can be made from the same description, we can create many objects from a class.

Classes are created using the keyword **class**. A class declaration defines a new type that links code and data. This new type is then used to declare objects of that class.

A class is a user defined data-type which has **data members** and **member functions**. Data members are the data variables and member functions are the functions used to manipulate these variables and together these data members and member functions defines the properties and behavior of the objects in a class.

A class member can be defined as **public**, **private** or **protected**. By default, members would be assumed as **private**.

In the UML, a class icon can be subdivided into compartments. The top compartment is for the name of the class, the second is for the variables of the class, and the third is for the methods of the class.



1.2. Class Name

By convention, the name of a user-defined class begins with a capital letter and, for readability, each subsequent word in the class name begins with a capital letter.

1.3. Data Members

Consider the attributes of some real-world objects:

- **RADIO** – station setting, volume setting.
- **CAR** – speedometer readings, amount of gas in its tank and what gear it is in.

These attributes form the data in our program. The values that these attributes take (the blue color of the petals, for example) form the state of the object.

1.4. Member Functions

Consider the operations of some real-world objects:

- **RADIO** – setting its station and volume (invoked by the person adjusting the radio's controls)
- **CAR** – accelerating (invoked by the driver), decelerating, turning and shifting gears.

These operations form the functions in program. Member functions define the class's behaviors.

1.5. Objects

In C++, when we define a variable of a class, we call it instantiating the class. The variable itself is called an instance of the class. A variable of a class type is also called an object. Instantiating a variable allocates memory for the object.

Syntax to define Object in C++:

```
ClassName objectVariableName;
```

```
Radio r;  
Car c;
```

1.6. Accessing Public Data Members

The **public** data members of objects of a class can be accessed using the direct member access operator (.). However, the **private** data members are not allowed to be accessed directly by the object. Accessing a data member depends solely on the access control of that data member.

1.7. Accessing Private Data Members (Setter & Getter Functions)

To access, use and initialize the **private** data member you need to create **getter** and **setter** functions, to get and set the value of the data member.

The **setter** function will set the value passed as argument to the **private** data member, and the **getter** function will return the value of the **private** data member to be used. Both **getter** and **setter** function must be defined **public**.

```
#include "iostream"  
using namespace std;  
  
class Student{  
  
    private:  
        int rollNo; // private data member  
  
    public:  
        // public function to get value of rollNo - getter  
        int getRollNo(){  
            return rollNo;  
        }  
        // public function to set value for rollNo - setter  
        void setRollNo(int i){
```



```

        rollNo = i;
    }
};

int main(){
    Student A;
    A.rollNo = 1;           // compile time error
    cout << A.rollNo;      // compile time error

    A.setRollNo(1);        // rollNo initialized to 1
    cout << A.getRollNo(); // output will be 1
}

```

```

#include<iostream>
using namespace std;

class Shape{

    // as private so object.height and object.width is inaccessible

    int height, width;
public:

    //setters
    void setHeight(int h){
        height = h;
    }

    void setWidth(int w){
        width = w;
    }

    //getters
    int getHeight(){
        return height;
    }

    int getWidth(){
        return width;
    }
};

int main(){
    Shape shape;

    // setters
    shape.setHeight(5);
    shape.setWidth(2);
}

```

```

// getters
cout << "The Height is : " << shape.getHeight() << endl;
cout << "The Width is : " << shape.getWidth() << endl;
// shape.height or shape.width wont work as they are private

return 0;
}

```

Sample Run	The Height is : 5 The Width is : 2
-------------------	---------------------------------------

1.8. Member Functions in Classes

There are 2 ways to define a member function:

1. Inside Class Definition
2. Outside Class Definition

1.8.1. Inside class definition

With an inline function, the compiler tries to expand the code in the body of the function in place of a call to the function.

Note that all the member functions defined inside the class definition are by default inline, but you can also make any non-class function inline by using keyword **inline** with them. Inline functions are actual functions, which are copied everywhere during compilation, like preprocessor macro, so the overhead of function calling is reduced.

1.8.2. Outside Class Definition

To define a member function outside the class definition we have to use the scope resolution **::** operator along with class name and function name.

```

#include <iostream>
using namespace std;

class car{
private:
    int car_number;
    char car_model[10];
public:
    void getdata(); //function declaration
    void showdata();
};

//Outside class function definition

void car::getdata(){

    cout<< "Enter car number: ";
    cin>>car_number;
    cout<< "Enter car model: ";
    cin>>car_model;
}

```

```
//Outside class function definition
void car::showdata(){

    cout<<"Car number is "<<car_number << endl;
    cout<<"Car model is "<<car_model;
}

// main function starts
int main(){

    car c1;
    c1.getdata();
    c1.showdata();

    return 0;
}
```

Sample Run

```
Enter car number: 9999
Enter car model: Sedan
Car number is 9999
Car model is Sedan
```

2. Structure vs Classes

By default, all structure fields are **public**, or available to functions (like the **main()** function) that are outside the structure. Conversely, all class fields are **private**. That means they are not available for use outside the class. When you create a class, you can declare some fields to be **private** and some to be **public**. For example, in the real world, you might want your **name** to be **public** knowledge but your **social_security_number**, **salary**, or **age** to be **private**.

3. Transformation from Procedural to Object-Oriented Programming

3.1. Procedural Approach

```
#include <iostream>
using namespace std;

double calculateBMI (double w, double h){

    return w/(h*h)*703;
}

string findStatus (double bmi){

    string status;
    if (bmi < 18.5)
    {
        status = "underweight";
    }
}
```

```

    else if (bmi < 25.0)
    {
        status = "normal"; // so on
    }
    return status;
}

int main(){

    double bmi, weight, height;
    string status;

    cout << "Enter weight in pounds: ";
    cin >> weight;
    cout << "Enter height in inches: ";
    cin >> height;

    bmi = calculateBMI(weight, height);

    cout << "Your BMI is " << bmi << endl;
    cout << "Your Status is " << findStatus(bmi);

    return 0;
}

```

Sample Run

```

Enter weight in pounds: 143
Enter height in inches: 69
Your BMI is 21.1151
Your Status is normal

```

3.2. Object-Oriented Approach

```

#include <iostream>
using namespace std;

class BMI{

    double weight, height, bmi;
    string status;

    public:
        void getInput(){

            cout << "Enter weight in pounds: ";
            cin >> weight;
            cout << "Enter height in inches: ";
            cin >> height;
        }

        double calculateBMI(){

```



```

        return weight / (height * height) * 703;
    }

    string findStatus (){
        if (bmi < 18.5)
        {
            status = "underweight";
        }
        else if (bmi < 25.0)
        {
            status = "normal"; // so on
        }
        return status;
    }
    void printStatus (){

        bmi = calculateBMI();

        cout << "Your BMI is " << bmi << endl;
        cout << "Your Status is " << findStatus();
    }
};

int main(){

    BMI bmi;
    bmi.getInput();
    bmi.printStatus();

    return 0;
}

```

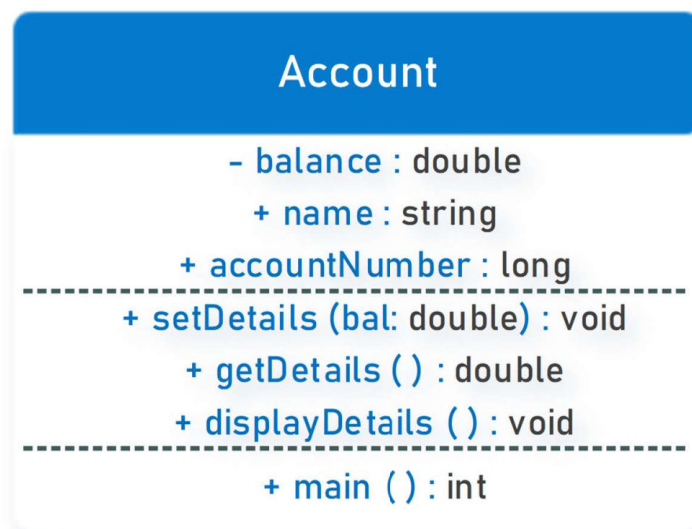
Sample Run

```

Enter weight in pounds: 143
Enter height in inches: 69
Your BMI is 21.1151
Your Status is normal

```

3.3. Object-Oriented Approach (another example with UML Diagram)



```
#include <iostream>
using namespace std;

class Account{
    private:

        double balance; // Account Balance

    public:

        string name; // Account Holder
        long accountNumber; // Account Number

        void setDetails(double bal){
            balance = bal;
        }

        double getDetails(){
            return balance;
        }

        void displayDetails(){
            cout << "Details are: " << endl;
            cout << "Account Holder: " << name << endl;
            cout << "Account Number: " << accountNumber << endl;
            cout << "Account Balance: " << getDetails();
        }
};

int main(){
```

Set and Get Functions to manipulate Private Data Members

	<pre> double accBal; Account currentAccount; currentAccount.getDetails(); cout << "Please enter the details" << endl; cout << "Enter Name: "; getline(cin, currentAccount.name); cout << "Enter Account Number: "; cin >> currentAccount.accountNumber; cout << "Enter Account Balance: "; cin >> accBal; currentAccount.setDetails(accBal); currentAccount.displayDetails(); return 0; } </pre> } Publicly Available Data Assigning values from. } Private Data Assigning private data using member function.
Sample Run	<pre> Please enter the details Enter Name: ABC Enter Account Number: 102225111 Enter Account Balance: 50000 Details are: Account Holder: ABC Account Number: 102225111 Account Balance: 50000 </pre>

4. Header Files

In C++, header files (**.h**) are used to declare functions, classes, and variables that can be shared across multiple source files. They help in organizing code, avoiding redundancy, and improving maintainability.

4.1. Essentials of Header Files

4.1.1. Purpose of Header Files

- Declares function prototypes
- Defines macros and constants
- Declares classes and structures
- Helps in modular programming

4.1.2. Header Guards

Header guards prevent multiple inclusions of the same header file, which can lead to redefinition errors.

4.1.3. Example of a Header Guard

```
#ifndef BMI_CALCULATOR_H // Check if BMI_CALCULATOR_H is not defined
#define BMI_CALCULATOR_H // Define BMI_CALCULATOR_H

class BMICalculator {
public:
    BMICalculator(double weight, double height);
    double calculateBMI();
    void displayBMI();
    std::string getBMICategory();
private:
    double weight;
    double height;
};

#endif // End of header guard
```

Alternatively, you can use #pragma once as a simpler method:

```
#pragma once
// Code inside the header file
```

4.2. Creating and Using a Header File

4.2.1. Creating a Header File

Create a .h file and declare functions, classes, or global variables.

Example: bmi_calculator.h

```
#ifndef BMI_CALCULATOR_H
#define BMI_CALCULATOR_H

#include <string>

class BMICalculator {
public:
    BMICalculator(double weight, double height);
    double calculateBMI();
    void displayBMI();
    std::string getBMICategory();
private:
    double weight;
    double height;
};

#endif // BMI_CALCULATOR_H
```

4.2.2. Implementing the Header File

Create a corresponding **.cpp** file to define the declared functions.

Example: bmi_calculator.cpp

```
#include "bmi_calculator.h"
#include <iostream>

BMICalculator::BMICalculator(double w, double h) : weight(w), height(h) {}

double BMICalculator::calculateBMI() {
    return weight / (height * height);
}

std::string BMICalculator::getBMICategory() {
    double bmi = calculateBMI();
    if (bmi < 18.5) return "Underweight";
    else if (bmi < 24.9) return "Normal weight";
    else if (bmi < 29.9) return "Overweight";
    else return "Obese";
}

void BMICalculator::displayBMI() {
    double bmi = calculateBMI();
    std::cout << "BMI: " << bmi << " (" << getBMICategory() << ")\n";
}
```

4.2.3. Using the Header File in Another Program

Include the header file in another C++ source file and use the class.

Example: main.cpp

```
#include <iostream>
#include "bmi_calculator.h"

int main() {
    double weight, height;
    std::cout << "Enter weight (kg): ";
    std::cin >> weight;
    std::cout << "Enter height (m): ";
    std::cin >> height;

    BMICalculator bmiCalc(weight, height);
    bmiCalc.displayBMI();

    return 0;
}
```

Compilation and Execution

```
g++ main.cpp bmi_calculator.cpp -o program
./program
```